MODELS OF COMPUTATION

CHAPTER 1

Given a problem, how do we find an efficient algorithm for its solution? Once we have found an algorithm, how can we compare this algorithm with other algorithms that solve the same problem? How should we judge the goodness of an algorithm? Questions of this nature are of interest both to programmers and to theoretically oriented computer scientists. In this book we shall examine various lines of research that attempt to answer questions such as these.

In this chapter we consider several models of a computer—the random access machine, the random access stored program machine, and the Turing machine. We compare these models on the basis of their ability to reflect the complexity of an algorithm, and derive from them several more specialized models of computation, namely, straight-line arithmetic sequences, bitwise computations, bit vector computations, and decision trees. Finally, in the last section of this chapter we introduce a language called "Pidgin ALGOL" for describing algorithms.

1.1 ALGORITHMS AND THEIR COMPLEXITY

Algorithms can be evaluated by a variety of criteria. Most often we shall be interested in the rate of growth of the time or space required to solve larger and larger instances of a problem. We would like to associate with a problem an integer, called the *size* of the problem, which is a measure of the quantity of input data. For example, the size of a matrix multiplication problem might be the largest dimension of the matrices to be multiplied. The size of a graph problem might be the number of edges.

The time needed by an algorithm expressed as a function of the size of a problem is called the *time complexity* of the algorithm. The limiting behavior of the complexity as size increases is called the *asymptotic time complexity*. Analogous definitions can be made for *space complexity* and *asymptotic space complexity*.

It is the asymptotic complexity of an algorithm which ultimately determines the size of problems that can be solved by the algorithm. If an algorithm processes inputs of size n in time cn^2 for some constant c, then we say that the time complexity of that algorithm is $O(n^2)$, read "order n^2." More precisely, a function $g(n)$ is said to be $O(f(n))$ if there exists a constant c such that $g(n) \leq cf(n)$ for all but some finite (possibly empty) set of nonnegative values for n.

One might suspect that the tremendous increase in the speed of calculations brought about by the advent of the present generation of digital computers would decrease the importance of efficient algorithms. However, just the opposite is true. As computers become faster and we can handle larger problems, it is the complexity of an algorithm that determines the increase in problem size that can be achieved with an increase in computer speed.

Suppose we have five algorithms A_1–A_5 with the following time complexities.

Algorithm	Time complexity
A_1	n
A_2	$n \log n$†
A_3	n^2
A_4	n^3
A_5	2^n

The time complexity here is the number of time units required to process an input of size n. Assuming that one unit of time equals one millisecond, algorithm A_1 can process in one second an input of size 1000, whereas algorithm A_5 can process in one second an input of size at most 9. Figure 1.1 gives the sizes of problems that can be solved in one second, one minute, and one hour by each of these five algorithms.

Algorithm	Time complexity	Maximum problem size		
		1 sec	1 min	1 hour
A_1	n	1000	6×10^4	3.6×10^6
A_2	$n \log n$	140	4893	2.0×10^5
A_3	n^2	31	244	1897
A_4	n^3	10	39	153
A_5	2^n	9	15	21

Fig. 1.1. Limits on problem size as determined by growth rate.

Algorithm	Time complexity	Maximum problem size before speed-up	Maximum problem size after speed-up
A_1	n	s_1	$10s_1$
A_2	$n \log n$	s_2	Approximately $10s_2$ for large s_2
A_3	n^2	s_3	$3.16s_3$
A_4	n^3	s_4	$2.15s_4$
A_5	2^n	s_5	$s_5 + 3.3$

Fig. 1.2. Effect of tenfold speed-up.

† Unless otherwise stated, all logarithms in this book are to the base 2.

Suppose that the next generation of computers is ten times faster than the current generation. Figure 1.2 shows the increase in the size of the problem we can solve due to this increase in speed. Note that with algorithm A_5, a tenfold increase in speed only increases by three the size of problem that can be solved, whereas with algorithm A_3 the size more than triples.

Instead of an increase in speed, consider the effect of using a more efficient algorithm. Refer again to Fig. 1.1. Using one minute as a basis for comparison, by replacing algorithm A_4 with A_3 we can solve a problem six times larger; by replacing A_4 with A_2 we can solve a problem 125 times larger. These results are far more impressive than the twofold improvement obtained by a tenfold increase in speed. If an hour is used as the basis of comparison, the differences are even more significant. We conclude that the asymptotic complexity of an algorithm is an important measure of the goodness of an algorithm, one that promises to become even more important with future increases in computing speed.

Despite our concentration on order-of-magnitude performance, we should realize that an algorithm with a rapid growth rate might have a smaller constant of proportionality than one with a lower growth rate. In that case, the rapidly growing algorithm might be superior for small problems, possibly even for all problems of a size that would interest us. For example, suppose the time complexities of algorithms A_1, A_2, A_3, A_4, and A_5 were really $1000n$, $100n \log n$, $10n^2$, n^3, and 2^n. Then A_5 would be best for problems of size $2 \leq n \leq 9$, A_3 would be best for $10 \leq n \leq 58$, A_2 would be best for $59 \leq n \leq 1024$, and A_1 best for problems of size greater than 1024.

Before going further with our discussion of algorithms and their complexity, we must specify a model of a computing device for executing algorithms and define what is meant by a basic step in a computation. Unfortunately, there is no one computational model which is suitable for all situations. One of the main difficulties arises from the size of computer words. For example, if one assumes that a computer word can hold an integer of arbitrary size, then an entire problem could be encoded into a single integer in one computer word. On the other hand, if a computer word is assumed to be finite, one must consider the difficulty of simply storing arbitrarily large integers, as well as other problems which one often avoids when given problems of modest size. For each problem we must select an appropriate model which will accurately reflect the actual computation time on a real computer.

In the following sections we discuss several fundamental models of computing devices, the more important models being the random access machine, the random access stored program machine, and the Turing machine. These three models are equivalent in computational power but not in speed.

Perhaps the most important motivation for formal models of computation is the desire to discover the inherent computational difficulty of various problems. We would like to prove lower bounds on computation time. In order to show that there is no algorithm to perform a given task in less than a certain

amount of time, we need a precise and often highly stylized definition of what constitutes an algorithm. Turing machines (Section 1.6) are an example of such a definition.

In describing and communicating algorithms we would like a notation more natural and easy to understand than a program for a random access machine, random access stored program machine, or Turing machine. For this reason we shall also introduce a high-level language called Pidgin ALGOL. This is the language we shall use throughout the book to describe algorithms. However, to understand the computational complexity of an algorithm described in Pidgin ALGOL we must relate Pidgin ALGOL to the more formal models. This we do in the last section of this chapter.

1.2 RANDOM ACCESS MACHINES

A random access machine (RAM) models a one-accumulator computer in which instructions are not permitted to modify themselves.

A RAM consists of a read-only input tape, a write-only output tape, a program, and a memory (Fig. 1.3). The input tape is a sequence of squares, each of which holds an integer (possibly negative). Whenever a symbol is read from the input tape, the tape head moves one square to the right. The output is a write-only tape ruled into squares which are initially all blank. When a write instruction is executed, an integer is printed in the square of the

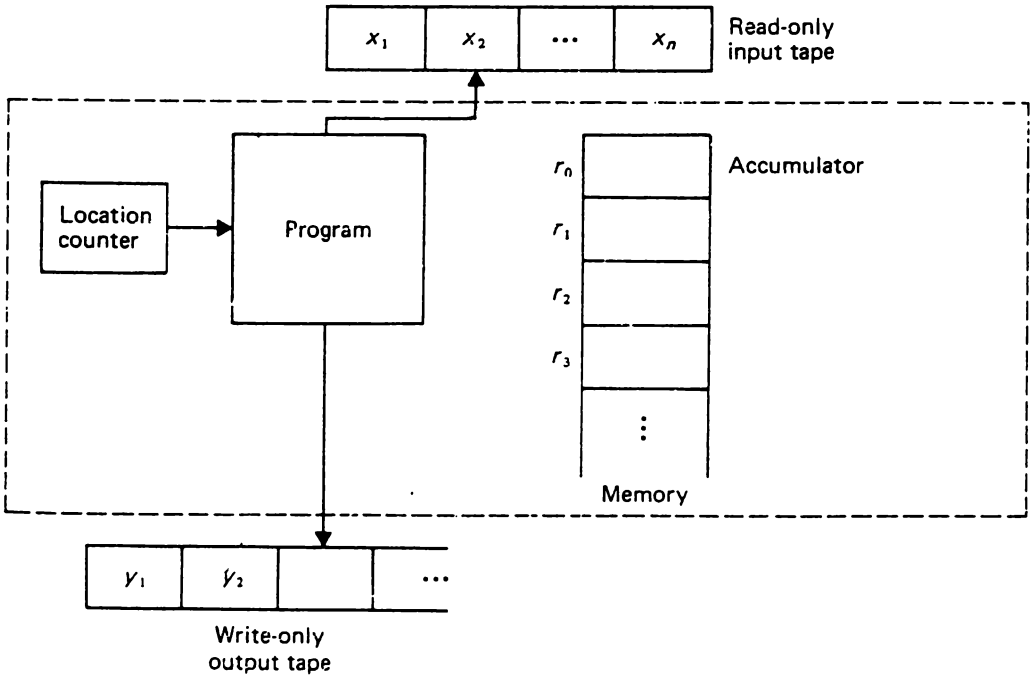

Fig. 1.3 A random access machine.

output tape that is currently under the output tape head, and the tape head is moved one square to the right. Once an output symbol has been written, it cannot be changed.

The memory consists of a sequence of registers, $r_0, r_1, \ldots, r_i, \ldots$, each of which is capable of holding an integer of arbitrary size. We place no upper bound on the number of registers that can be used. This abstraction is valid in cases where:

1. the size of the problem is small enough to fit in the main memory of a computer, and
2. the integers used in the computation are small enough to fit in one computer word.

The program for a RAM is not stored in the memory. Thus we are assuming that the program does not modify itself. The program is merely a sequence of (optionally) labeled instructions. The exact nature of the instructions used in the program is not too important, as long as the instructions resemble those usually found in real computers. We assume there are arithmetic instructions, input-output instructions, indirect addressing (for indexing arrays, e.g.) and branching instructions. All computation takes place in the first register r_0, called the *accumulator*, which like every other memory register can hold an arbitrary integer. A sample set of instructions for the RAM is shown in Fig. 1.4. Each instruction consists of two parts—an *operation code* and an *address*.

In principle, we could augment our set with any other instructions found in real computers, such as logical or character operations, without altering the order-of-magnitude complexity of problems. The reader may imagine the instruction set to be so augmented if it suits him.

Operation code	Address
1. LOAD	operand
2. STORE	operand
3. ADD	operand
4. SUB	operand
5. MULT	operand
6. DIV	operand
7. READ	operand
8. WRITE	operand
9. JUMP	label
10. JGTZ	label
11. JZERO	label
12. HALT	

Fig. 1.4. Table of RAM instructions.

An operand can be one of the following:

1. $=i$, indicating the integer i itself.
2. A nonnegative integer i, indicating the contents of register i.
3. $*i$, indicating indirect addressing. That is, the operand is the contents of register j, where j is the integer found in register i. If $j \leq 0$, then the machine halts.

These instructions should be quite familiar to anyone who has programmed in assembly language. We can define the meaning of a program P with the help of two quantities, a mapping c from nonnegative integers to integers and a "location counter" which determines the next instruction to execute. The function c is a *memory map;* $c(i)$ is the integer stored in register i (the *contents* of register i).

Initially, $c(i) = 0$ for all $i \geq 0$, the location counter is set to the first instruction in P, and the output tape is all blank. After execution of the kth instruction in P, the location counter is automatically set to $k + 1$ (i.e., the next instruction), unless the kth instruction is JUMP, HALT, JGTZ, or JZERO.

To specify the meaning of an instruction we define $v(a)$, *the value of operand* a, as follows:

$$\begin{aligned} v(=i) &= i, \\ v(i) &= c(i), \\ v(*i) &= c(c(i)). \end{aligned}$$

The table of Fig. 1.5 defines the meaning of each instruction in Fig. 1.4. Instructions not defined, such as STORE $=i$, may be considered equivalent to HALT. Likewise, division by zero halts the machine.

During the execution of each of the first eight instructions the location counter is incremented by one. Thus instructions in the program are executed in sequential order until a JUMP or HALT instruction is encountered, a JGTZ instruction is encountered with the contents of the accumulator greater than zero, or a JZERO instruction is encountered with the contents of the accumulator equal to zero.

In general, a RAM program defines a mapping from input tapes to output tapes. Since the program may not halt on all input tapes, the mapping is a partial mapping (that is, the mapping may be undefined for certain inputs). The mapping can be interpreted in a variety of ways. Two important interpretations are as a function or as a language.

Suppose a program P always reads n integers from the input tape and writes at most one integer on the output tape. If, when $x_1, x_2, \ldots, x_n$ are the integers in the first n squares of the input tape, P writes y on the first square of the output tape and subsequently halts, then we say that P computes the function $f(x_1, x_2, \ldots, x_n) = y$. It is easily shown that a RAM, like any other

Instruction	Meaning
1. LOAD a	$c(0) \leftarrow v(a)$
2. STORE i	$c(i) \leftarrow c(0)$
STORE $*i$	$c(c(i)) \leftarrow c(0)$
3. ADD a	$c(0) \leftarrow c(0) + v(a)$
4. SUB a	$c(0) \leftarrow c(0) - v(a)$
5. MULT a	$c(0) \leftarrow c(0) \times v(a)$
6. DIV a	$c(0) \leftarrow \lfloor c(0)/v(a) \rfloor$†
7. READ i	$c(i) \leftarrow$ current input symbol.
READ $*i$	$c(c(i)) \leftarrow$ current input symbol. The input tape head moves one square right in either case.
8. WRITE a	$v(a)$ is printed on the square of the output tape currently under the output tape head. Then the tape head is moved one square right.
9. JUMP b	The location counter is set to the instruction labeled b.
10. JGTZ b	The location counter is set to the instruction labeled b if $c(0) > 0$; otherwise, the location counter is set to the next instruction.
11. JZERO b	The location counter is set to the instruction labeled b if $c(0) = 0$; otherwise, the location counter is set to the next instruction.
12. HALT	Execution ceases.

† Throughout this book, $\lceil x \rceil$ (*ceiling* of x) denotes the least integer equal to or greater than x, and $\lfloor x \rfloor$ (*floor*, or *integer part* of x) denotes the greatest integer equal to or less than x.

Fig. 1.5. Meaning of RAM instructions. The operand a is either $=i$, i, or $*i$.

reasonable model of a computer, can compute exactly the *partial recursive functions*. That is, given any partial recursive function f we can define a RAM program that computes f, and given any RAM program we can define an equivalent partial recursive function. (See Davis [1958] or Rogers [1967] for a discussion of recursive functions.)

Another way to interpret a RAM program is as an acceptor of a language. An *alphabet* is a finite set of symbols, and a *language* is a set of strings over some alphabet. The symbols of an alphabet can be represented by the integers $1, 2, \ldots, k$ for some k. A RAM can accept a language in the following manner. We place an input string $s = a_1 a_2 \cdots a_n$ on the input tape, placing the symbol a_1 in the first square, the symbol a_2 in the second square, and so on. We place 0, a symbol we shall use as an endmarker, in the $(n + 1)$st square to mark the end of the input string.

The input string s is *accepted* by a RAM program P if P reads all of s and the endmarker, writes a 1 in the first square of the output tape, and halts. The *language accepted by* P is the set of accepted input strings. For input strings not in the language accepted by P, P may print a symbol other than 1 on the output tape and halt, or P may not even halt. It is easily shown that a language is accepted by a RAM program if and only if it is *recursively enumerable*. A language is accepted by a RAM that halts on all inputs if and only if it is a *recursive* language (see Hopcroft and Ullman [1969] for a discussion of recursive and recursively enumerable languages).

Let us consider two examples of RAM programs. The first defines a function; the second accepts a language.

Example 1.1. Consider the function $f(n)$ given by

$$f(n) = \begin{cases} n^n & \text{for all integers } n \geq 1, \\ 0 & \text{otherwise.} \end{cases}$$

A Pidgin ALGOL program which computes $f(n)$ by multiplying n by itself $n-1$ times is illustrated in Fig. 1.6.† A corresponding RAM program is given in Fig. 1.7. The variables $r1$, $r2$, and $r3$ are stored in registers 1, 2, and 3 respectively. Certain obvious optimizations have not been made, so the correspondence between Figs. 1.6 and 1.7 will be transparent. □

```
begin
    read r1;
    if r1 ≤ 0 then write 0
    else
        begin
            r2 ← r1;
            r3 ← r1 − 1;
            while r3 > 0 do
                begin
                    r2 ← r2 * r1;
                    r3 ← r3 − 1
                end;
            write r2
        end
end
```

Fig. 1.6. Pidgin ALGOL program for n^n.

† See Section 1.8 for a description of Pidgin ALGOL.